

PySpark Mastery Guide

Essential DataFrame Transformations for Q401 & Q402 LLD Assessments

To succeed in your PySpark assessments, you must master "pure functions"—functions that take a DataFrame as input, apply standard PySpark transformations, and return a new DataFrame (or specific Python types). This guide breaks down the critical PySpark methods you will need.

1. Adding and Modifying Columns

`.withColumn(colName, col)`

This is your primary tool for creating new derived columns or replacing existing ones. It evaluates an expression for every row.

- **Arguments:** `colName` (string) - the name of the new column, `col` (Column expression) - the logic to calculate the value.
- **Use Cases:** Computing gross amount, delivery days, total activity, or usage scores.

```
# Example: add_gross_amount(df) from pyspark.sql.functions import col
df_new = df.withColumn("gross_amount", col("quantity") * col("unit_price")) #
Example: flag_on_time_delivery(df, max_days) df_new =
df.withColumn("is_on_time", col("delivery_days") <= max_days)
```

2. Filtering Data

`.filter(condition) / .where(condition)`

Used to select rows that meet specific criteria. Both methods are functionally identical in PySpark.

- **Arguments:** A condition expression returning a boolean (e.g., `col("category") == 'Electronics'`).
- **Use Cases:** Filtering returned orders, keeping only roaming events, or finding high-value orders.

```
# Example: filter_returned_orders(df) df_filtered =
df.filter(col("returned") == "Y") # Example: high_value_orders(df, threshold)
df_filtered = df.filter(col("net_amount") > threshold) # Example: Multiple
conditions (use & for AND, | for OR) df_filtered = df.filter((col("latency_ms")
> 100) & (col("roaming") == "Y"))
```

3. Aggregations and Grouping

`.groupBy(*cols).agg(*exprs)`

The core mechanism for answering "per category", "by region", or "daily trend" questions. You group rows that share the same values in specified columns, then apply aggregate functions like `sum()`, `avg()`, or `count()`.

- **Arguments:** `groupBy()` takes column names. `agg()` takes PySpark aggregate functions.
- **Important:** Always use `.alias()` inside your aggregation to rename the resulting column exactly as the LLD requires.

```
from pyspark.sql.functions import sum, avg, count # Example:
revenue_by_category(df) df_agg = df.groupBy("category").agg(
sum("net_amount").alias("total_revenue") ) # Example:
dropped_call_rate_by_plan(df) df_agg = df.groupBy("plan").agg(
count("*").alias("total_calls"), sum("dropped_calls").alias("dropped_total") )
```

4. Sorting and Limiting

`.orderBy(*cols)`

Sorts the DataFrame. By default, it sorts in ascending order. Use `.desc()` on a column object for descending order.

- **Tie-Breaking:** When instructions say "tie-break by user_id asc", you must provide multiple arguments to `orderBy`.

`.limit(n)`

Returns only the first `n` rows of a DataFrame. Almost always used immediately after an `orderBy` to get "Top N" results.

- **Argument:** An integer `n`.

```
# Example: top_n_customers_by_spend(df, n) df_top = df.groupBy("customer_id")
\ .agg(sum("net_amount").alias("total_spend")) \ .orderBy(
col("total_spend").desc(), # Primary sort col("customer_id").asc() # Tie-breaker )
\ .limit(n)
```

5. Conditional Logic and Null Handling

`when(condition, value).otherwise(value)`

PySpark's equivalent of an IF-ELSE statement. Often used inside `withColumn` or inside aggregations to count specific conditions.

```
from pyspark.sql.functions import when, concat_ws # Example:
return_rate_by_category logic # Counting only rows where returned == 'Y'
df_returns = df.groupBy("category").agg( count("*").alias("total_cnt"),
sum(when(col("returned") == 'Y', 1).otherwise(0)).alias("returned_cnt") ) #
Example: enrich_full_name (Q402) # If full_name is null, combine first_name and
last_name. Otherwise, keep it. df_enriched = df.withColumn( "full_name",
when(col("full_name").isNull(), concat_ws(" ", col("first_name"),
col("last_name"))) .otherwise(col("full_name")) )
```

6. Joins

`.join(other, on, how)`

Combines two DataFrames based on a common key. By default, it performs an inner join.

```
# Example: join_profile_plan(profiles_df, plans_df) # Joining on the
common column "plan_id" df_joined = profiles_df.join(plans_df, on="plan_id",
how="left")
```

7. Date and Time Functions

Data preprocessing often involves converting strings to dates or calculating durations.

Function	Purpose	Usage Example
<code>to_date(col)</code>	Converts string to DateType	<code>withColumn("date", to_date(col("date_str")))</code>
<code>datediff(end, start)</code>	Difference in days	<code>withColumn("days", datediff(col("delivery"), col("ship")))</code>
<code>unix_timestamp(col)</code>	Date/Timestamp to epoch seconds	<code>withColumn("epoch", unix_timestamp(col("event_date")))</code>

8. Strategy: Returning Specific Output Types

Not all LLD functions return a DataFrame. Some require you to extract specific primitive types (string, int, tuple, list). You must process the DataFrame and then pull the data out to Python memory using `.collect()` or `.first()`.

Extracting a Tuple (e.g., `best_selling_product`)

If you need to return (product_id, total_qty):

```
def best_selling_product(df): df_agg =  
df.groupBy("product_id").agg(sum("quantity").alias("total_qty")) df_sorted =  
df_agg.orderBy(col("total_qty").desc(), col("product_id").asc()) # Extract the  
first row as a PySpark Row object first_row = df_sorted.first() # Return as a  
Python tuple return (first_row["product_id"], first_row["total_qty"])
```

Extracting a List (e.g., `list_channels` or `list_cities`)

If you need to return a sorted Python List[str]:

```
def list_channels(df): # Get distinct values, order them, and collect  
back to driver rows =  
df.select("channel").distinct().orderBy("channel").collect() # rows is a list  
of Row objects. Extract the string value using list comprehension. return  
[row["channel"] for row in rows]
```

Defining a StructType (e.g., `define_schema`)

When loading a CSV safely, you explicitly define its structure using StructType and StructField.

```
from pyspark.sql.types import StructType, StructField, StringType,  
IntegerType, DoubleType def define_plan_schema(): return StructType([  
StructField("plan_id", StringType(), True), StructField("plan_name",  
StringType(), True) ])
```

Final Checklist for Assessments: